

**THE UNIVERSITY OF AZAD JAMMU & KASHMIR,
MUZAFFARABAD**

Name

Fatima Arshad

Roll No

2024-SE-24

Semester

04

Course Title

DBMS

Submitted By

Engr. Awais Rathore

Lab

02

Database: University Database

1. Introduction

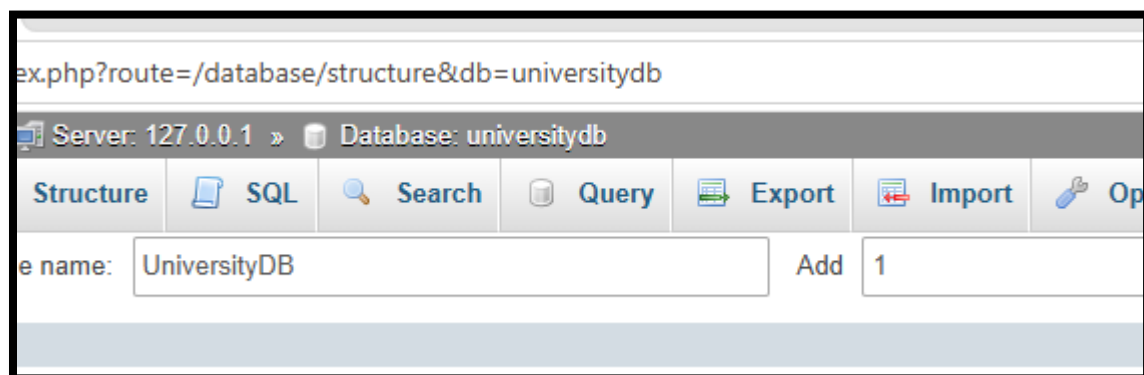
This lab focuses on building a University Database (UniversityDB) that manages students, courses, and enrollments. It also demonstrates how to perform data definition (DDL) and data manipulation (DML) operations such as creating tables, inserting records, altering table structures, updating data, deleting records, truncating tables, and dropping tables. The purpose of this lab is to understand how relational databases handle relationships, constraints, and modifications in a real-world academic scenario.

2. Database Creation

We begin by creating the database and selecting it for use: sql

```
CREATE DATABASE UniversityDB;  
USE UniversityDB;
```

This ensures that all subsequent tables and data are stored inside UniversityDB.



Server: 127.0.0.1 > Database: universitydb

Structure SQL Search Query Export Import Operations Privileges Routines Events Trig

Filters

Containing the word:

Table	Action	Rows	Type	Collation	Size	Overhead
☐ courses	★ Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_general_ci	16.0 KiB	-
☐ enrollments	★ Browse Structure Search Insert Empty Drop	3	InnoDB	utf8mb4_general_ci	32.0 KiB	-
☐ students	★ Browse Structure Search Insert Empty Drop	2	InnoDB	utf8mb4_general_ci	80.0 KiB	-
3 tables	Sum	8	InnoDB	utf8mb4_general_ci	128.0 KiB	0 B

☐ Check all
 With selected:

Print Data dictionary

3. Students Table

The Students table stores essential student information.

- **student_id** is the primary key and auto-incremented.
- **roll_number** and email are unique to prevent duplication.
- **name** stores the student's full name.

✓ Showing rows 0 - 1 (2 total, Query took 0.0004 seconds.)

`SELECT * FROM `students``

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	student_id	roll_number	full_name	email
☐ Edit Copy Delete	1	R001	Ali Khan	ali@gmail.com
☐ Edit Copy Delete	2	R002	Ahmed	ahmed@gmail.com

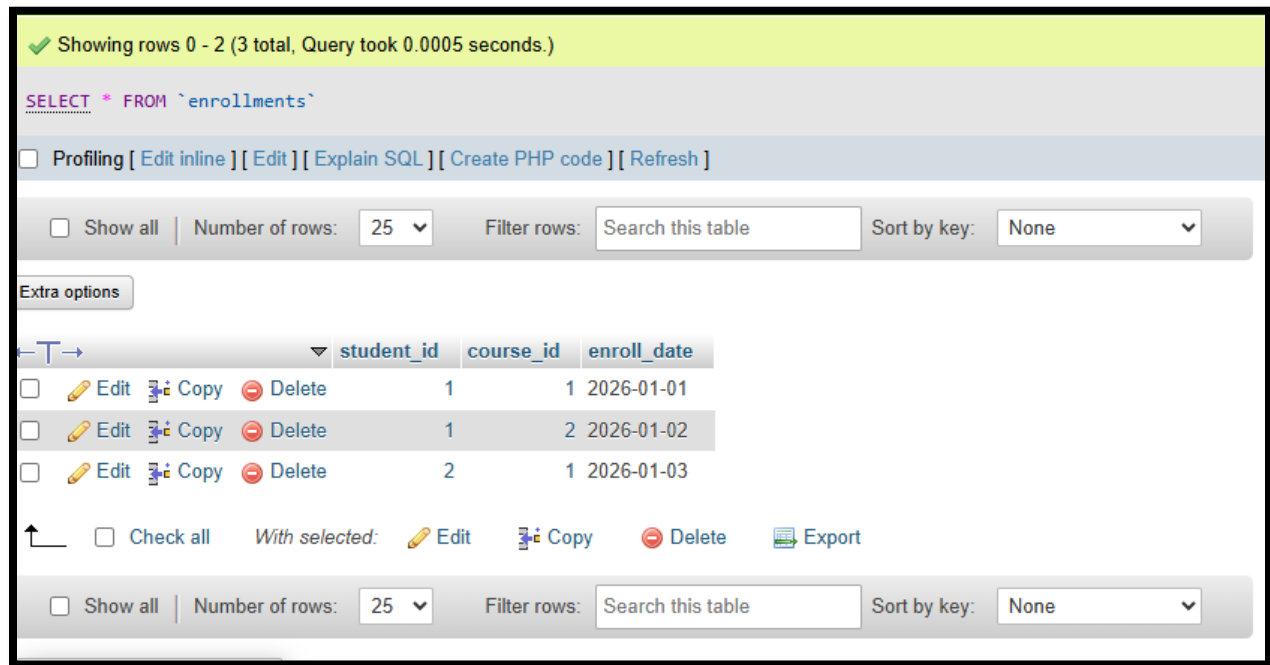
☐ Check all | With selected: Edit Copy Delete Export

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

4. Enrollments Table

The Enrollments table connects students with courses.

- **Composite primary key** (studentid, courseid) ensures that a student cannot enroll in the same course twice.
- **Foreign keys** maintain referential integrity with Students and Courses.



	student_id	course_id	enroll_date
☐ Edit Copy Delete	1	1	2026-01-01
☐ Edit Copy Delete	1	2	2026-01-02
☐ Edit Copy Delete	2	1	2026-01-03

5. Courses Table

The Courses table stores the list of courses offered by the university.

- Each course has a unique **course_id** as the primary key.

✓ Showing rows 0 - 2 (3 total, Query took 0.0004 seconds.)

```
SELECT * FROM `courses`
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

☐ Show all | Number of rows: 25 ▼ Filter rows: Sort by key: None ▼

Extra options

	course_id	course_name
☐ Edit Copy Delete	1	Math
☐ Edit Copy Delete	2	Science
☐ Edit Copy Delete	3	English

↑ ☐ Check all With selected: [Edit](#) [Copy](#) [Delete](#) [Export](#)

☐ Show all | Number of rows: 25 ▼ Filter rows: Sort by key: None ▼

Server: 127.0.0.1 » Database: universitydb » Table: enrollments

[Browse](#) [Structure](#) [SQL](#) [Search](#) [Insert](#) [Export](#) [Import](#) [Privileges](#) [Operations](#) [Tracking](#) [Tools](#)

[Table structure](#) [Relation view](#)

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐ 1	student_id	int(11)			No	None			Change Drop More
☐ 2	course_id	int(11)			No	None			Change Drop More
☐ 3	enroll_date	date			Yes	NULL			Change Drop More

↑ ☐ Check all With selected: [Browse](#) [Change](#) [Drop](#) [Primary](#) [Unique](#) [Index](#) [Spatial](#) [Fulltext](#)

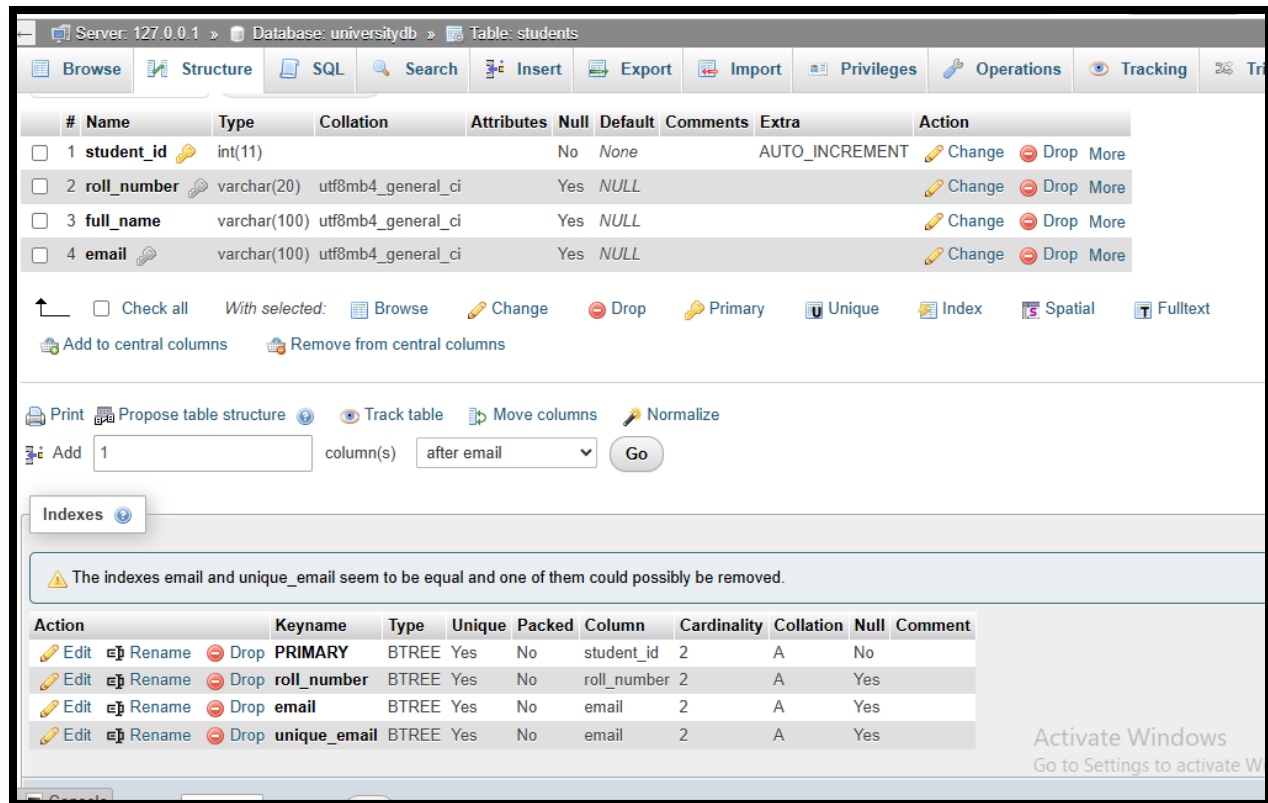
[Add to central columns](#) [Remove from central columns](#)

[Print](#) [Propose table structure](#) [Track table](#) [Move columns](#) [Normalize](#)

[Add](#) 1 column(s) after enroll_date [Go](#)

[Indexes](#)

Action	Keyname	Type	Unique	Packed	Column	Cardinality	Collation	Null	Comment
Edit Rename Drop	PRIMARY	BTREE	Yes	No	student_id	3	A	No	
Edit Rename Drop					course_id	3	A	No	
Edit Rename Drop	course_id	BTREE	No	No	course_id	3	A	No	



6. Altering the Students Table

We performed several structural changes to the Students table:

- **Add column:** Added age column.
- **Modify column:** Made age column NOT NULL.
- **Rename column:** Changed name to full_name.
- **Drop column:** Removed the age column.
- **Add unique constraint:** Ensured that email remains unique.

These operations demonstrate how a table can evolve over time to meet new requirements.

✓ Showing rows 0 - 1 (2 total, Query took 0.0004 seconds.)

```
SELECT * FROM Students;
```

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

	student_id	roll_number	full_name	email
☐ Edit Copy Delete	1	R001	Ali Khan	ali@gmail.com
☐ Edit Copy Delete	2	R002	Ahmed	ahmed@gmail.com

↑ ☐ Check all | With selected: Edit Copy Delete Export

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

7. Data Manipulation

- We executed several commands to manipulate data:
- **Select all students:** Displayed all student records.
- **Update student name:** Changed Ali's name to Ali Khan.
- **Delete student:** Removed Sara's record.
- **Truncate enrollments:** Cleared all enrollment records while keeping the table structure.
- **Drop courses table:** Completely removed the Courses table.

These commands show how to manage and maintain data in a live database.

Bookmark this SQL query

✓ 0 rows affected. (Query took 0.0017 seconds.)

```
UPDATE Students SET full_name='Ali Khan' WHERE student_id=1;
```

[Edit inline] [Edit] [Create PHP code]

✓ 0 rows deleted. (Query took 0.0007 seconds.)

```
DELETE FROM Students WHERE student_id=3;
```

[Edit inline] [Edit] [Create PHP code]

Error

SQL query: [Copy](#)

```
DROP TABLE Courses;
```

MySQL said: 

#1451 - Cannot delete or update a parent row: a foreign key constraint fails

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0004 seconds.)

```
SELECT * FROM `enrollments`
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

student_id course_id enroll_date

Query results operations

 Create view

 Bookmark this SQL query

Label:

☐ Let every user access this bookmark

Bookmark this SQL query

Error

SQL query: [Copy](#)

```
DROP TABLE Courses;
```

MySQL said: 

```
#1451 - Cannot delete or update a parent row: a foreign key constraint fails
```

Exporting tables from "universitydb" database

Export templates:

New template:

Existing templates:

Template name

Template name

Create

Template:

-- Select a template --

Update

Delete

Export method:

☒ Quick - display only the minimal options

☐ Custom - display all possible options

Format:

SQL

Export

Activate Windows

8. ER Diagram Explanation

The Entity-Relationship (ER) Diagram for UniversityDB illustrates the logical structure:

- **Students** → **Enrollments**: One student can enroll in many courses (1:N).
- **Courses** → **Enrollments**: One course can have many students (1:N).
- Enrollments acts as a bridge table, creating a many-to-many relationship between Students and Courses.

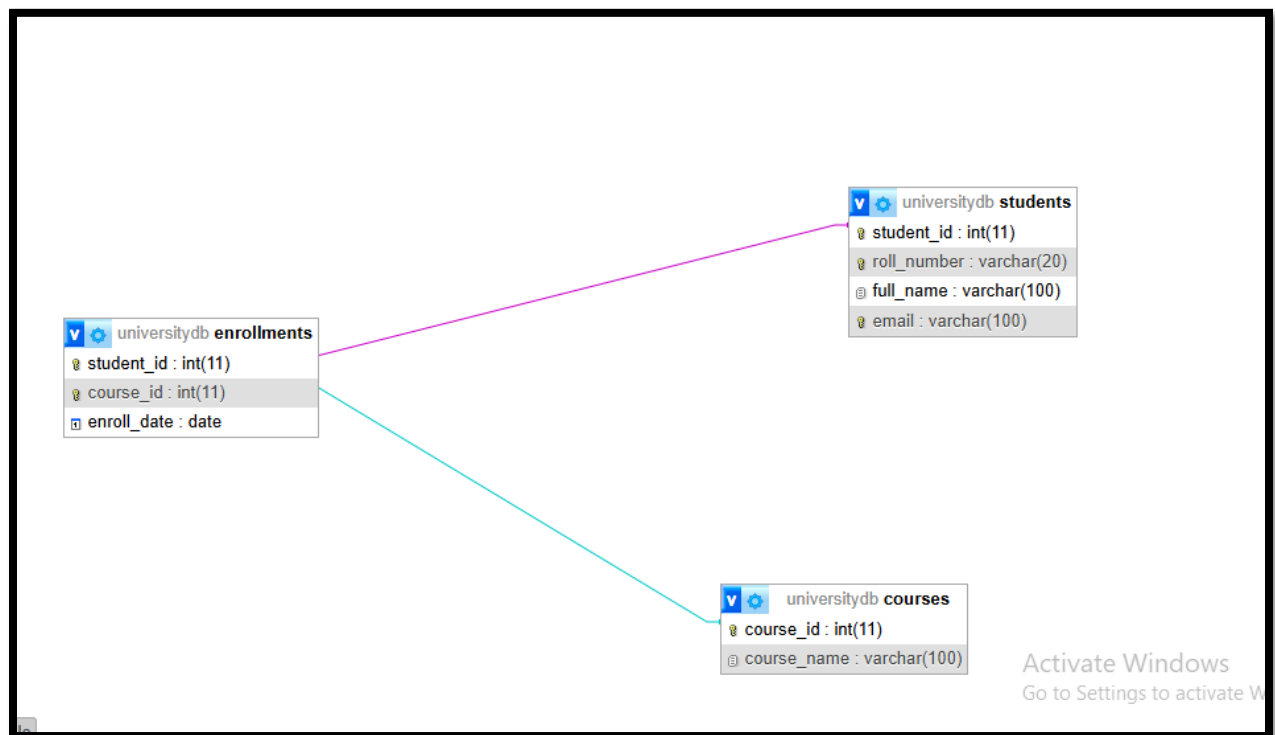
➤ ER Diagram (Conceptual Description)

Entities: Students, Courses, Enrollments

Relationships:

- **Students** ↔ Enrollments (1:N)
- **Courses** ↔ Enrollments (1:N)
- **Overall: Students** ↔ Courses (M:N through Enrollments)

This diagram clearly shows how students and courses are connected in the university system.



9. Conclusion

This lab demonstrates how to design and manage a University Database. We created tables, inserted data, and applied constraints to ensure data integrity. We also practiced altering table structures, updating records, deleting data, truncating tables, and dropping tables. The ER diagram highlights the many-to-many relationship between students and courses, which is a common scenario in academic databases.
